

Sistemas Operacionais

Gerência de Memória

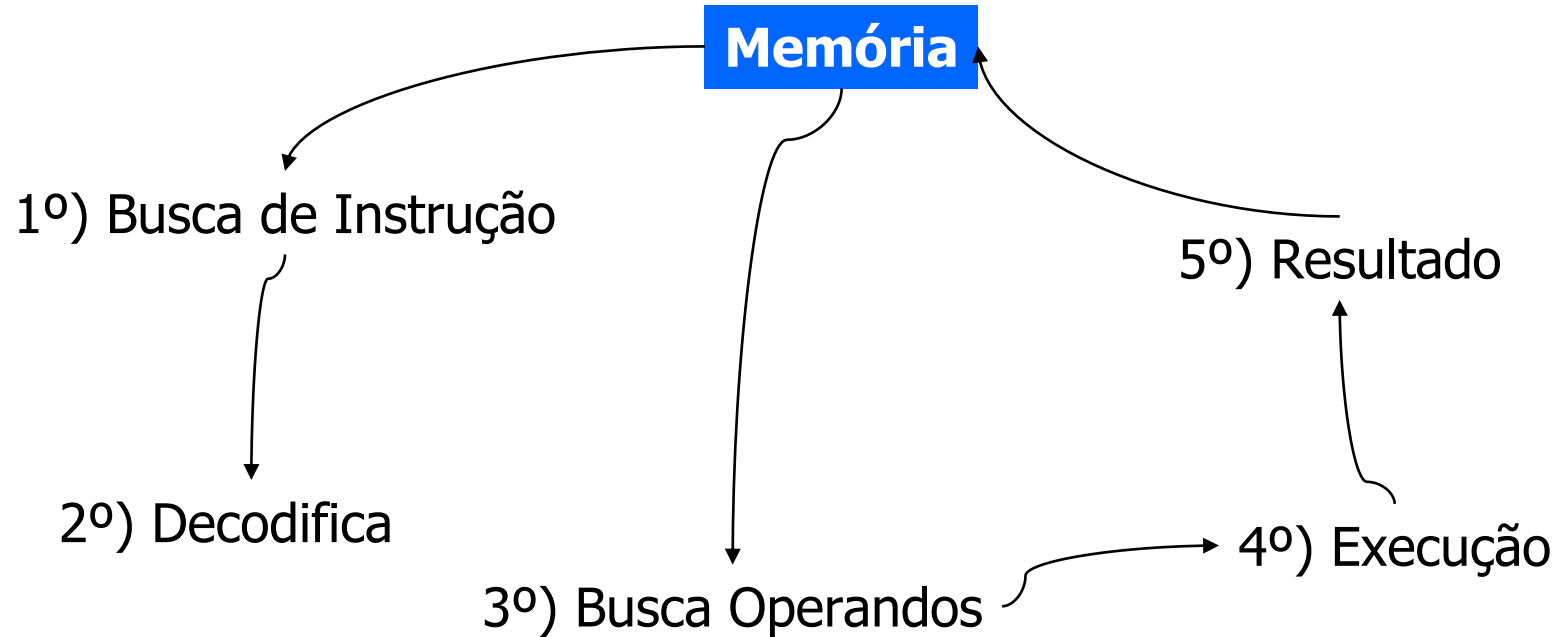
Introdução

- Multiprogramação
 - Vários processos prontos para execução, concorrem à CPU
 - Para que a troca de contexto seja rápida, os processos devem estar na memória principal
- Gerência de Memória do SO
 - deve prover mecanismos para que os processos utilizem a memória de forma segura e eficiente
 - há várias técnicas de gerência
 - cada SO emprega uma técnica em particular e isto depende fortemente da arquitetura do computador

Introdução

- Memória
 - Memória = vetor de palavras ou bytes com seus endereços
 - A CPU busca instruções da memória, de acordo com o valor do Contador de Programa (podem ser de busca ou armazenamento em endereços de memória específicos)
- Gerência de Memória
 - Todo endereço de memória gerado pela CPU deve ser verificado em termos de legalidade e possivelmente ser mapeado em um endereço físico.
 - A verificação não pode ser implementada de forma eficiente no software -> limitada pelo hardware

Ciclo de Execução de Instrução



- A Unidade de Memória vê apenas um fluxo de endereços de memória (não sabe como são gerados ou para que servem)
- Observar: sequência de endereços de memória gerados pelo programa em execução

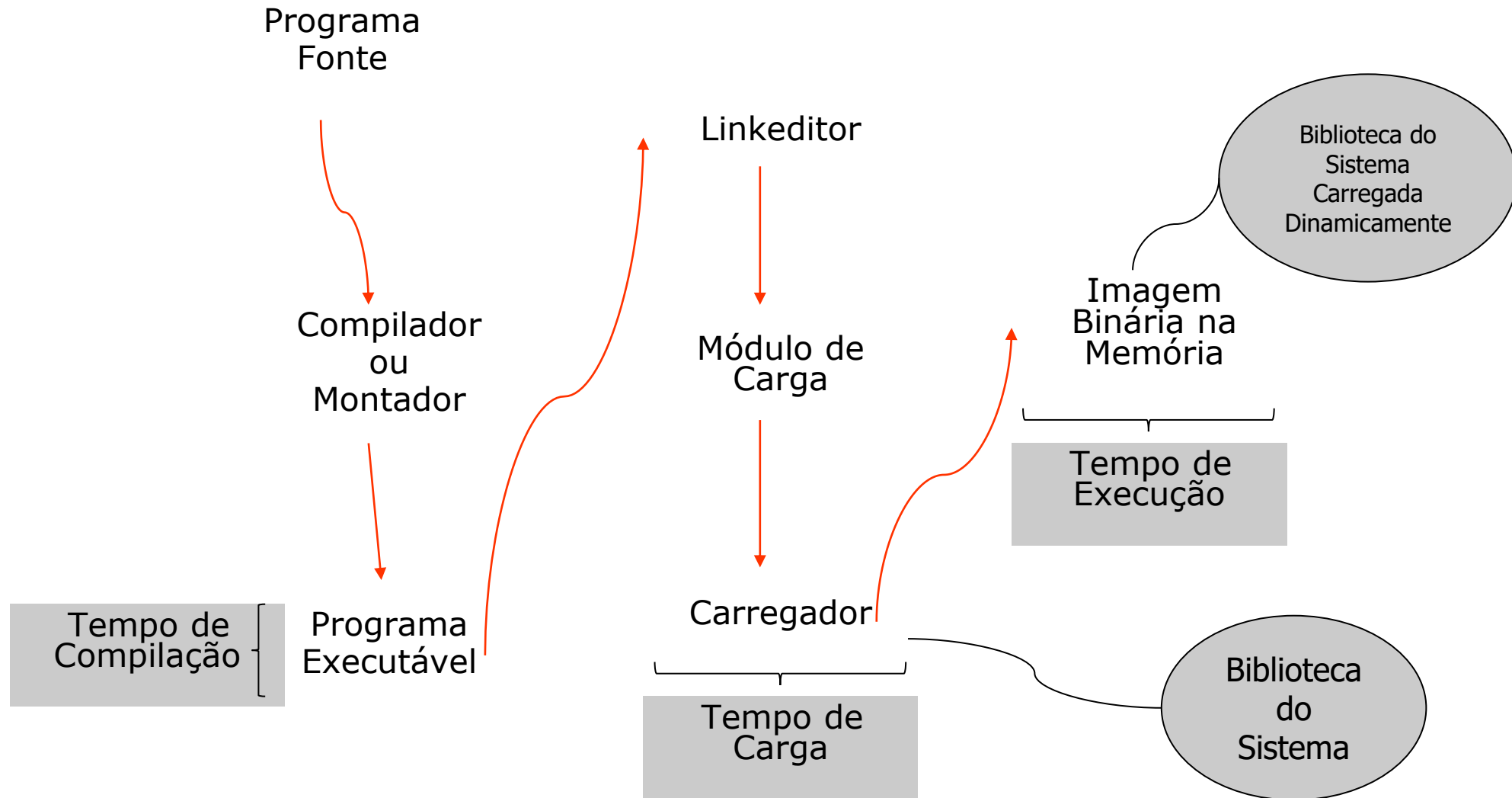
Mapeamento de Endereços

- Fila de Entrada = coleção de processos no disco esperando para serem carregados na memória
- Procedimento normal
 - Selecionar um dos processos na Fila de Entrada e carregá-lo na memória.
Função do escalonador de Longo Prazo
 - A medida em que um processo é executado, ele acessa instruções e dados da memória. Ao terminar, seu espaço de memória é disponibilizado.

Mapeamento de Endereços (2)

- O espaço de endereços no computador começa em 00000, mas o primeiro endereço de um processo de usuário não precisa ser 00000
- Isto afeta os endereços que o programa do usuário vai usar
- Um programa de usuário passa por várias etapas antes de ser executado:

Mapeamento de Endereços (3)



Mapeamento de Endereços (4)

- Os endereços podem ser representados de diferentes formas durante essas etapas e mapeados também em momentos diferentes.
- Cada associação é um mapeamento de um espaço de endereço para outro e pode ser feita em etapas diferentes

Mapeamento de Endereços (5)

- 1 - Em tempo de compilação
 - Se sabe-se antecipadamente que um processo de usuário reside na posição X da memória, o código compilado iniciará nessa posição e se estende a partir daí.
 - Se a posição de início mudar, é necessário recompilar o código.
 - Isso se chama Código Absoluto
 - Carregador absoluto: os endereços do processo não são alterados na carga

Mapeamento de Endereços (6)

- 2 - Em tempo de carga
 - O compilador deve gerar um código relocável (se não há código absoluto). A associação final dos endereços é retardada até o instante de carga.
 - Se o endereço de início mudar, os endereços do processo são corrigidos pelo carregador para que o programa execute corretamente no local onde foi carregado.
 - Carregador relocador

Mapeamento de Endereços (7)

- 3 - Em tempo de execução
 - Se o processo durante sua execução pode ser movido de um segmento de memória para outro, a associação dos endereços é feita em tempo de execução.
 - A maioria dos Sistemas Operacionais de uso geral usa esse método. É necessário um hardware de uso geral para que isso funcione.
 - Esse hardware faz o mapeamento de endereços lógicos para endereços físicos

Espaço de Endereçamento Lógico x Físico

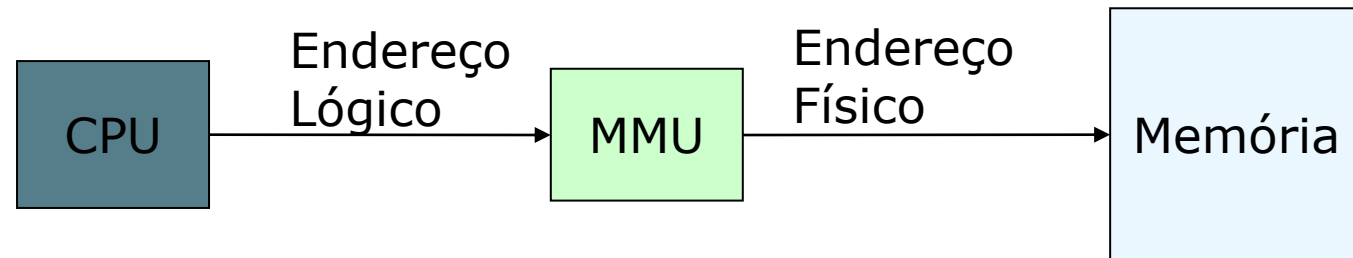
- Memória Lógica de um processo
 - é aquela que o processo enxerga
 - os endereços manipulados pelo processo são Endereços Lógicos
 - Por exemplo, as variáveis de um processo contém endereços lógicos.
 - Cada processo possui sua memória lógica, independente da memória lógica de outros processos
 - Espaço de Endereçamento Lógico (ou Virtual) de um processo é o conjunto de endereços lógicos que esse processo pode endereçar (enxergar). Há um espaço de endereçamento lógico por processo.

Espaço de Endereçamento Lógico x Físico (2)

- Memória Física
 - é aquela implementada pelos circuitos integrados de memória
 - o Endereço Físico é usado para endereçar os circuitos integrados
 - Espaço de Endereçamento Físico é formado por todos os endereços aceitos pelos circuitos integrados de memória.

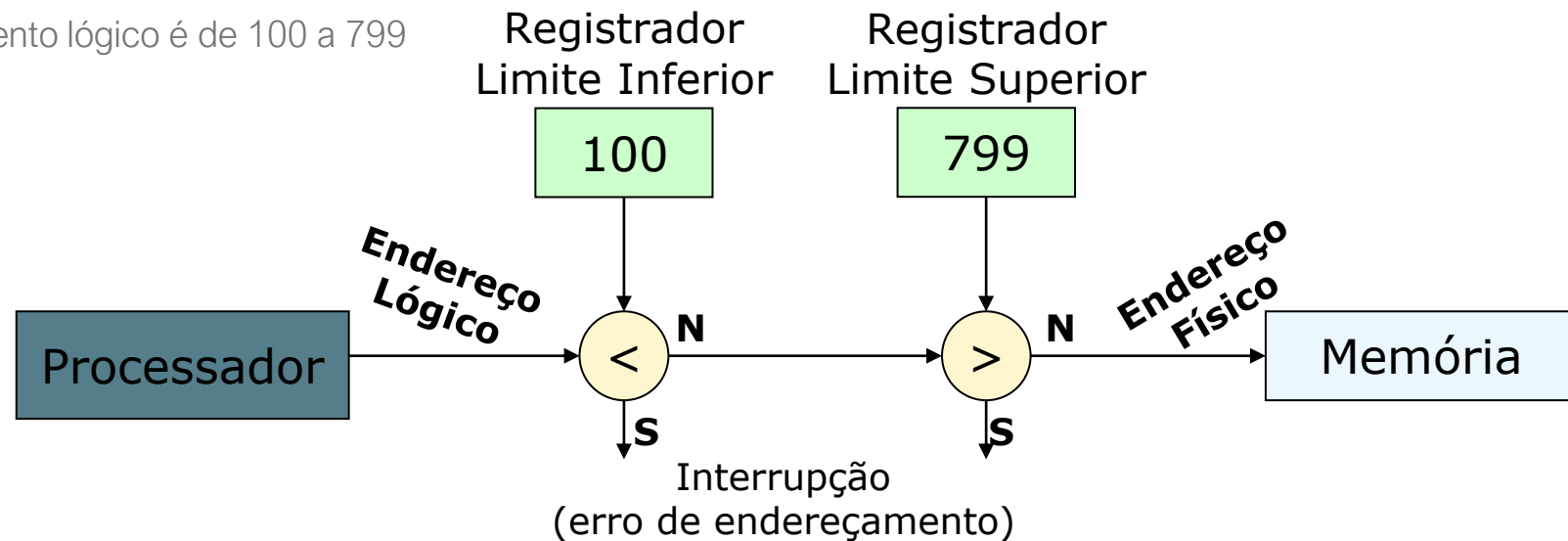
Espaço de Endereçamento Lógico x Físico (3)

- Os mapeamentos de endereços em tempo de compilação e em tempo de carga resultam em endereçamentos lógicos e físicos iguais. O mapeamento de endereços em tempo de execução, não.
- O Mapeamento de tempo de execução dos endereços lógicos para físicos é feito pela Unidade de Gerência de Memória (MMU – Memory Management Unit)
- A MMU é um componente do hardware



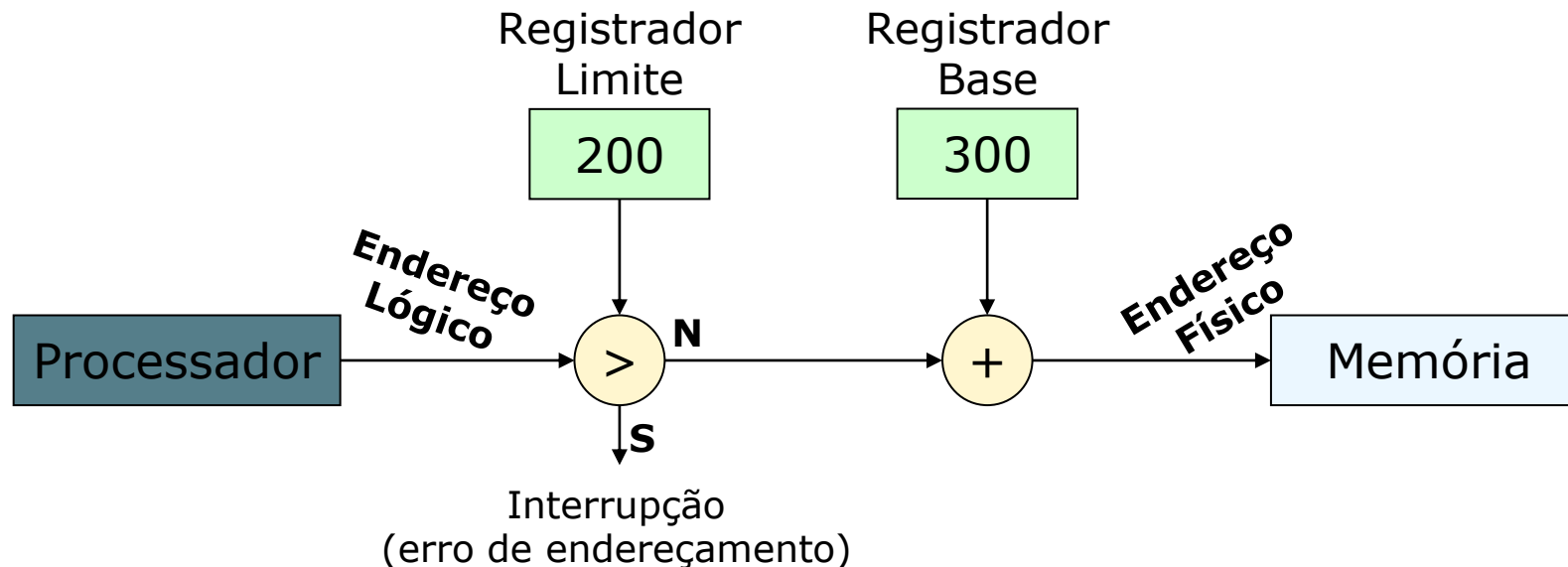
Proteção de Memória

- Realizada pela MMU
- Exemplo com Registradores de Limite
 - Considera que os endereços lógicos e físicos são idênticos
 - O conteúdo dos Registradores Limite define o espaço de endereçamento lógico do processo
 - Neste exemplo, o espaço de endereçamento lógico é de 100 a 799



Proteção de Memória

- Exemplo com Registrador Limite e Registrador Base
 - Considera que os endereços lógicos e físicos são diferentes
 - O espaço de endereçamento lógico do processo vai de zero até o Registrador Limite
 - A área de memória física do processo inicia no valor do Registrador Base e tem o tamanho da memória lógica do processo (Limite)



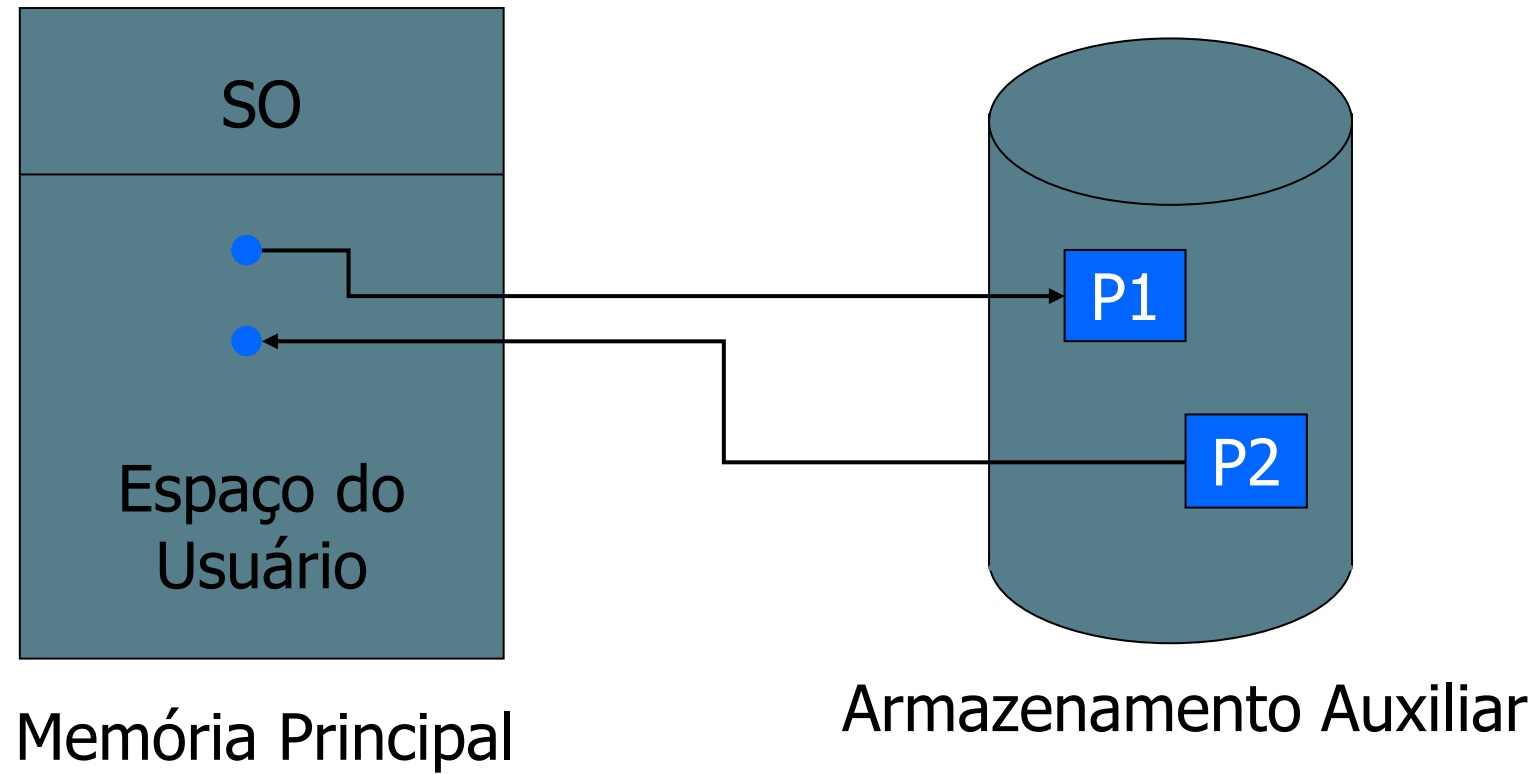
Proteção de Memória

- Registradores base e limite
 - devem ser protegidos-> acessados somente no modo supervisor
 - Fazem parte do contexto do processo. Devem ser mantidos no PCB do processo.
 - Troca de contexto: os valores dos registradores são copiados do PCB para os registradores da MMU

Swapping

- Troca = Swapping
- Um processo pode ser removido da memória temporariamente para um armazenamento auxiliar e retomado à memória para continuar a execução
- Liberar memória

Swapping (2)



Swapping (3)

- Geralmente, um processo que é descarregado será carregado no mesmo espaço de memória que ocupava anteriormente
- Isso é determinado pelo método de resolução de endereço
 - Se for na carga ou montagem
 - processo não pode ser movido para posições diferentes
 - Se for em tempo de execução
 - é possível passar para posições diferentes porque endereços físicos são calculados em tempo de execução

Swapping (4)

- Troca requer armazenamento auxiliar
 - disco rápido e grande o suficiente para acomodar cópias de todas as imagens de memória para os usuários e fornecer acesso direto a essas imagens na memória
- O Sistema Operacional mantém Fila de Processos Prontos
 - processos cujas imagens de memória estão no dispositivo de armazenamento auxiliar ou na memória principal prontos para executar

Swapping (5)

- Sempre que o escalonador de CPU buscar um processo, chama o dispatcher para fazer a troca de contexto:
 - Verifica se o próximo processo na fila está na memória
 - Se não houver região de memória livre, descarrega um processo de memória (swap out)
 - Carrega o processo desejado em seu lugar (swap in)
 - Em seguida, recarrega os registradores e transfere o controle para o processo selecionado.

Swapping (6)

- O tempo da troca de contexto com swapping é relativamente alto (transferência do processo: memória-> disco-> memória)
 - Processo de usuário = 1 MB
 - Taxa de Transferência do disco rígido = 5 MB / s
 - Para 1 MB -> 200 ms + latência (8 ms) = 208 ms
 - Carga e descarga = $208 * 2 = 416$ ms

Swapping (7)

- O tempo de execução deve ser longo em relação ao tempo de troca
 - No RR, quantum $> 0,416$ segundos para esse caso
- A parte principal do tempo de troca é o tempo de transferência (que é proporcional a quantidade de memória trocada)
- É útil saber quanto de memória cada processo está usando

Swapping (8)

- Para descarregar um processo é necessário ter certeza que está completamente inativo
- Os sistemas operacionais geralmente possuem um espaço em disco para swap, separado do espaço para os arquivos.